1 Verilog Implementation

```
module phase5(
  input clk,
  input rst,
  input [1:0] code_in,
  output reg phase5_done,
  output reg phase5_fail,
  output reg all_done=1'b0
);

  parameter safe = 3'b000;
  parameter s1   = 3'b001;
  parameter s2   = 3'b010;
  parameter s3   = 3'b011;
  parameter done = 3'b101;
  parameter fail = 3'b110;

  reg alaram;
  parameter TIMEOUT_LIMIT = 100_000_000;

  reg [2:0] current_state, next_state;
  reg [26:0] timeout_counter;
  reg [2:0] prev_state;

  always @(posedge clk) begin
    if (rst) begin
      current_state <= safe;
      timeout_counter <= 0;
      prev_state <= safe;
    end else begin
      if ((current_state == prev_state) && (current_state != safe) && (current_state != done) && (current_state != fail))
        timeout_counter <= timeout_counter + 1;
      else
        timeout_counter <= 0;

      if (timeout_counter >= TIMEOUT_LIMIT)
        current_state <= fail;
      else
        current_state <= next_state;

      prev_state <= current_state;
    end
  end

  always @(*) begin
    next_state = current_state;
    phase5_done = 0;
    phase5_fail = 0;
    alaram = 0;
```

```
    case(current_state)
      safe:
        next_state = s1;

      s1:
        if (code_in == 2'b01)
          next_state = s2;
        else
          next_state = fail;

      s2:
        if (code_in == 2'b10)
          next_state = s3;
        else
          next_state = fail;

      s3:
        if (code_in == 2'b11)
          next_state = done;
        else
          next_state = fail;

      done: begin
        phase5_done = 1;
        all_done=1;
      end

      fail: begin
        phase5_fail = 1;
        alaram = 1;
      end

      default:
        next_state = safe;
    endcase
  end

endmodule
```